**Overview.** My research interests lie in making FPGA-based SmartNICs easier to program while preserving their performance advantages. Through developing datacenter systems on both SoC-based and FPGA-based SmartNICs, I have seen how hardware offloading can improve performance and how much effort it can take to implement each new offload. In my Ph.D., I hope to investigate how developers can express protocol-specific behavior using reusable hardware components while reducing the need for custom hardware logic.

**Research Experiences.** During my M.S. at UNIST, I worked with Prof. Youngbin Im on FlexServer[1], a research project focused on optimizing content servers that deliver files from storage to clients over the network. Reading files from storage can stall the server because the process must wait until the file data has been loaded into the TCP send buffer before continuing with other work. To address this, I implemented a transmission system built on multi-tier caching that prefetches hot files from NVMe SSDs into host DRAM and SmartNIC memory. I implemented this system on both BlueField and Agilio SmartNICs. A key challenge was preserving correct TCP behavior while adhering to a zero-copy design. I reorganized the TCP send buffer to handle data according to its location: requesting the SmartNIC to transmit data stored in its memory, passing physical addresses to the NIC for data in host memory, and using an optimized I/O engine to read data from NVMe SSDs into the TCP send buffer. FlexServer achieved up to 351 Gbps, outperforming the state-of-the-art baseline, IO-TCP[2], by 4.4×. I am the first author of the resulting paper. However, the pipeline still used a bounce buffer to stage data from NVMe devices before network transmission. FPGA-based SmartNICs could offer further opportunities to optimize this data path, such as eliminating the staging step through peer-to-peer DMA between NVMe devices and the NIC. I became interested in FPGA-based SmartNICs because of this flexibility.

At MangoBoost, I worked on software stacks for NVMe/TCP accelerators built on FPGA-based SmartNICs. As a coauthor of BoostX-NTI[3], accepted at ISCA 2026, I developed its PCIe interface and NVMe-compliant control plane. Since managing complex state in hardware is challenging, we ran the control plane on an onboard Arm processor to handle NVMe register configuration and admin commands, while keeping performance-critical NVMe I/O processing in FPGA logic. A key challenge was coordinating these components to preserve correct NVMe behavior during network faults, for which I developed fault-handling mechanisms. The accelerator exposed remote NVMe/TCP storage as a local NVMe/PCIe device, supporting Windows and Linux hosts and earning UNH-IOL certification[4]. In this project, hardware offloading focused on NVMe read and write commands with a simple data path: transferring data via DMA using host memory addresses. Had we used BlueField, data would have needed to be staged in Arm memory before transmission by the NIC. However, this flexibility came at a cost: development was much slower than on SoC-based SmartNICs.

**Motivation.** These experiences drew my attention to how FPGA-based SmartNICs could be developed more efficiently. Even for the simple data path in BoostX-NTI, rebuilding the FPGA logic after adding a new feature often took a day or two, while the control plane software I developed on the Arm processor could be rebuilt and tested within minutes.

FPGA shells, such as Coyote[5], Coyote v2[6], and $\mu$Shell[7], provide a foundation for reusing hardware components, including network stacks. I want to go further by enabling developers to express user-defined behavior through *protocol adapters* executed by a runtime on pre-built hardware. This requires a compiler that translates adapters into programs for this execution substrate and a runtime that coordinates their interactions with reusable hardware components. While my company focuses on developing dedicated hardware components for its products, I want to study compiler and runtime support that would allow developers to express new behavior without building these components each time.

***Reusable Hardware Components* and *Protocol Adapters*.** Hardware implementations of network stacks and I/O functionality, such as a TOE, can serve as shared building blocks across different offloads. I want to investigate how developers can reuse these implementations while expressing protocol- and role-specific behavior through *protocol adapters*. For example, an NVMe/TCP initiator or target could be organized into NVMe functionality, a TCP stack, and initiator- or target-specific protocol logic. The initiator logic posts submission commands and handles completions, while the target logic interprets incoming commands, coordinates NVMe operations, and generates responses. The NVMe and TCP implementations could serve as *reusable hardware components*. However, the initiator and target still require dedicated hardware logic, which is time-consuming and demanding to implement.

To reduce this implementation effort, I envision *protocol adapters* that express role-specific behavior by coordinating *reusable hardware components* and responding to protocol messages and events. Dagger[8] shows how application-defined RPC interfaces can be supported by an FPGA-based offloading system, and motivates me to develop an abstraction for protocol-specific behavior beyond RPC processing. Unlike Dagger, which executes RPC handlers on the host CPU, I want to support message-handling logic on the FPGA through *protocol adapters*. Developers would specify how protocol messages and events trigger operations on and interactions between these components. I envision a runtime that executes *protocol adapters* on a pre-built hardware execution substrate, allowing developers to change protocol behavior without resynthesizing the FPGA design, as long as the required hardware components are already available. As an initial example, I would build an NVMe/TCP initiator adapter that posts NVMe submission commands and handles completions over TCP without dedicated hardware logic, and compare it against the hand-built implementation in BoostX-NTI to measure what the abstraction costs. I want to understand which protocol behaviors can be expressed on a fixed execution substrate and how adapter execution affects latency, throughput, and resource use.

In the long term, I hope to apply *protocol adapters* to LLM inference, where KV caches move between nodes in prefill–decode disaggregation[9;10] and are offloaded from GPU memory to CPU memory and SSDs[10]—data paths that would otherwise each require dedicated hardware logic on an FPGA-based SmartNIC.

**Why Cornell. Prof. Jiaxin Lin**'s work on SmartNIC systems aligns with my interest in making hardware offloading easier to program. I found Alkali[11] particularly interesting because it enables a single program to run efficiently across different SmartNICs by accounting for the characteristics of each hardware target, including FPGAs. I hope to pursue research in this direction with Prof. Lin. I am also interested in working with **Prof. Zhiru Zhang** on flexible hardware systems such as Dagger[8], as well as FPGA-based acceleration of AI systems.

My long-term goal is to lead research that turns new programming abstractions for hardware offloading into deployed datacenter systems, whether in academia or at an industrial research lab. Both paths require defining problems from first principles rather than following product roadmaps, and I hope to develop that ability through a Ph.D.

**AI Use Disclosure.** I used ChatGPT to check grammar and spelling in my own writing throughout the application. I also used it to review how sentences in this statement of purpose might be interpreted and to identify potential ambiguities.

# References

[1] Dongu Kim, Seyeong Park, Seyeon Kim, Sangtae Ha, and Youngbin Im. FlexServer: A High-Performance Content Server Leveraging Asynchronous I/O and SmartNICs. Preprint, 2024.

[2] Taehyun Kim, Deondre Martin Ng, Junzhi Gong, Youngjin Kwon, Minlan Yu, and KyoungSoo Park. Rearchitecting the TCP Stack for I/O-Offloaded Content Delivery. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023.

[3] Hamin Jang, Jinha Jeong, Wonseok Lee, Jun Heo, Jongcheon Lee, Hyunjae Chu, Taehyun Kim, Youngwoo Jeong, Dongu Kim, Heetaek Jeong, Changsu Kim, Dongup Kwon, and Jangwoo Kim. BoostX-NTI: Fast, Scalable and Flexible Storage Architecture with NVMe/TCP Initiator Acceleration. In *Proceedings of the 53rd ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2026. doi: 10.1109/ISCA66397.2026.00081.

[4] University of New Hampshire InterOperability Laboratory. NVMe Integrators List, n.d. URL `https://www.iol.unh.edu/registry/nvme`.

[5] Dario Korolija, Timothy Roscoe, and Gustavo Alonso. Do OS abstractions make sense on FPGAs? In *Proceedings of OSDI*, 2020.

[6] Benjamin Ramhorst, Dario Korolija, Maximilian Jakob Heer, Jonas Dann, Luhao Liu, and Gustavo Alonso. Coyote v2: Raising the Level of Abstraction for Data Center FPGAs. In *Proceedings of SOSP*, 2025.

[7] Jiyang Chen, Anubhav Panda, Harshavardhan Unnibhavi, Atsushi Koshiba, and Pramod Bhatotia. $\mu$Shell: A Microkernel-based FPGA Shell Architecture. In *Proceedings of OSDI*, 2026.

[8] Nikita Lazarev, Shaojie Xiang, Neil Adit, Zhiru Zhang, and Christina Delimitrou. Dagger: Efficient and Fast RPCs in Cloud Microservices with Near-Memory Reconfigurable NICs. In *Proceedings of ASPLOS*, 2021.

[9] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 193–210, 2024.

[10] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025.

[11] Jiaxin Lin, Zhiyuan Guo, Mihir Shah, Tao Ji, Yiying Zhang, Daehyeok Kim, and Aditya Akella. Enabling Portable and High-Performance SmartNIC Programs with Alkali. In *Proceedings of NSDI*, 2025.